

LLM Architecture

Sunday, February 15, 2026 10:28 PM

Oracle AI Agent

Building an AI agent is easy

Building an enterprise-grade AI agent that is:

- scalable,
- secure,
- governed,
- maintainable,
- and production-ready

1. Design AI Agents Around Business Outcomes

Good Enterprise Approach

Business Problem	AI Agent Goal
Invoice delays	Detect overdue invoices
HR ticket overload	Automate employee queries
Procurement approvals	Intelligent approval routing
Supplier compliance	Risk detection

2. Keep AI Agents Task-Oriented

Recommended Pattern

Agent	Responsibility
HR Agent	Employee queries
Procurement Agent	Invoice workflows
Finance Agent	Expense approvals
IT Support Agent	Ticket handling

3. Use Tools for Real-Time Enterprise Data

4. Use Oracle Native Fusion APIs Wherever Possible

5. Keep Prompts Modular

Prompt Type	Purpose
System Prompt	Agent behavior
Tool Prompt	API interaction
Validation Prompt	Safety checks
Summarization Prompt	Formatting output

6. Minimize Token Consumption

Tokens directly impact:

- Cost
- Latency
- Scalability

Good Prompt

Summarize invoice status in 3 bullets.

Bad Prompt

Provide a complete detailed explanation regarding every invoice and related information available in the system.

7. Choose the Right LLM for the Right Task

Use Case	Model Type
FAQ chatbot	Small fast model
Financial reasoning	Advanced reasoning model
Workflow automation	Deterministic model
Creative generation	Generative model

8. Design for Determinism in Enterprise Workflows

9. Implement Human-in-the-Loop (HITL)

LLM Architecture

The architecture of LLM models is the **internal structure** and design principles that govern how these models process text data. In simple terms, it's how an LLM is built: what components and layers it contains, how they interact, and how information flows through the model.

From a business perspective, understanding the architecture helps in three ways:

- It clarifies **resource requirements**, such as **computing power** and **storage**.
- It defines how easily an LLM can be **trained**, **fine-tuned**, or **integrated** into existing **workflows**.
- It highlights limitations and strengths, aiding in better **vendor selection** or build-vs-buy decisions.

Every large language model is a **combination of architectural components** that work together to **transform raw text into informative output**. These elements are not unique to any one model but are shared across most LLM implementations.

Prompt: " What are the flight options to Newyork, travelling from Hyderabad ?" -> **Vocabulary Dictionary -> Lookup Table**

Tokens : "What","are","the"..... " What" ,"A" ,"re" , "th" ,"e"

Token = What - 523 (Byte Pair Encoding - BPE) - Split into Tokens using Tokenzier , WordPiece, SentencePiece -> 523 unique identifier ,"are" - 782 ,"the" - 19

Print(names[2])

Tokens : [523,782,19]

Token ID - Embedding Vector

Vocabulary Size ; 50000

Embedding Dimension: 782

Token ID : (VocabularySize * Embedding Dimension) = **dense floating-vector**

Neural Networks - Numbers, Vectors, Matrices

Embedding("king") - Embedding("man")+Embedding("woman") = Embedding("queen")

Token = I - Dictionary Key

Embedding = E - Numerical Representation

TokenVector =E[i]

Word - Huge 3d

King-man+woman=queen

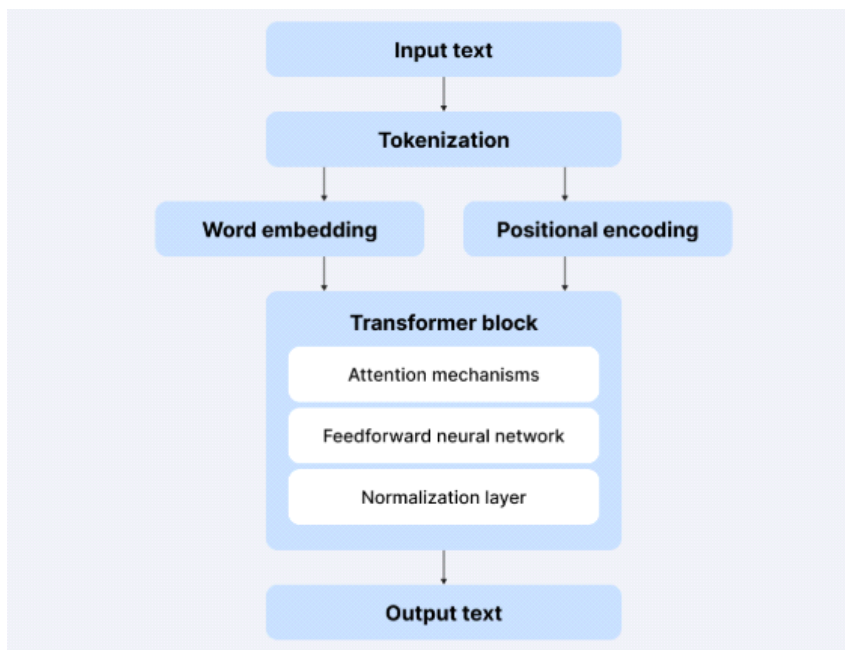
Car -
Democracy

Linked List - Node

Node [Data+Next Node]
[Prev Node - Data - Next Node]

Temperature

Low - (0.1 to 0.3) - More factual answers, produce Good outputs
Medium (0.70) - Balanced
High (1.7 above) - Creative, But may produce nonsense tokens



Basic LLM Architecture

Types of LLM Architecture

Type	Main function	Best for	Examples
Encoder-only	Understanding input	Text classification, search, NER, sentiment	BERT
Decoder-only (Causal)	Generating output from input	Text generation, chat, code completion	GPT-3, GPT-4
Encoder-decoder	Transforming input into new output	Translation, summarization, Q&A	T5, BART
Prefix decoder	Task-specific generation with prompt	Instruction following, few-shot learning	FLAN-T5, GPT-3 (prompted)